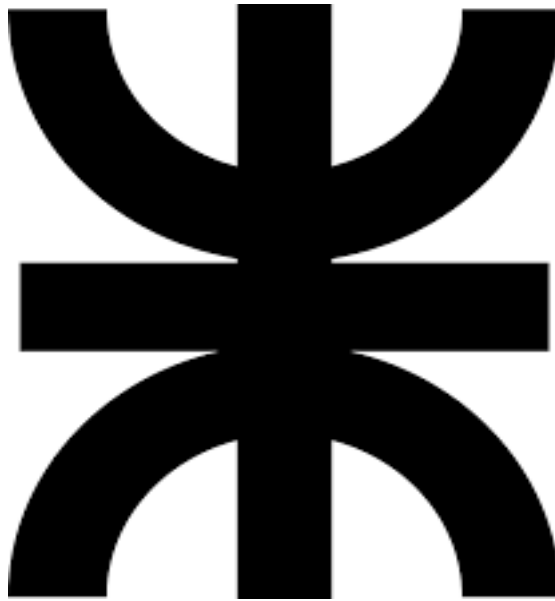


UNIDAD 2 - DESAFÍO 2



UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RESISTENCIA

Materia: Sistema de Gestión de Base de Datos.

Comisión: K 3.1

Año: 2026

Grupo: 6

Integrantes:

- Basualdo Alvarez, Facundo F.
- Godoy, Marianela Yasmin.
- Pare, Collante Agustina.
- Sanchez, Maria.
- Sanchez, Romina.
- Toledo, Francisco

Docentes:

- Prof. Andrés Fantin.
- JTP. Fernández, Juan Carlos.
- Aux. Edgar Nahuel Dominguez.

0 · Verificación del entorno

Se ejecutó 00_instalar_lab_postgresql.sql, que crea la tabla heap y la tabla particionada equivalente. La salida confirma motor, versión, cantidad de filas y cantidad de particiones.

```
ANALYZE
version
-----
PostgreSQL 18.6 (Ubuntu 18.6-1.pgdg24.04+2) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, 64-bit
(1 fila)

filas_heap
-----
        600000
(1 fila)

filas_particionadas
-----
        600000
(1 fila)

particiones
-----
         48
(1 fila)
postgres=# |
```

Evidencia 1 — Versión del motor, filas en heap y particionada, y cantidad de particiones.

A continuación se fijó el esquema de trabajo antes de cualquier medición:

```
SET search_path TO sgbd_a2_lab, public;
```

```
postgres=# SET search_path TO sgbd_a2_lab, public;
SET
postgres=# |
```

Evidencia 2 — Esquema de trabajo fijado en la sesión de psql.

1 · Operación e hipótesis

Consulta prioritaria

Histórico de un cliente (consulta A) sobre pedido_part, sin filtro por fecha:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT pedido_id, fecha, estado, total
FROM pedido_part
WHERE cliente_id = 1500
ORDER BY fecha DESC
LIMIT 20;
```

```
SELECT pedido_id, fecha, estado, total
FROM pedido_part
WHERE cliente_id = 1500
ORDER BY fecha DESC
LIMIT 20;
```

Evidencia 3 — Consulta prioritaria y parámetros exactos utilizados.

Hipótesis (redactada antes de ejecutar el DDL)

El particionamiento por fecha no aporta pruning a una consulta filtrada únicamente por cliente_id: el planificador se ve obligado a recorrer las 48 particiones con Seq Scan y a materializar un Sort antes del LIMIT.

Esperamos que, al crear un índice compuesto local B-tree sobre (**cliente_id, fecha DESC**), cada partición acceda directamente a las tuplas del cliente vía Index Scan, el nodo Sort desaparezca (el orden lo provee el

árbol B) y la lectura corte de forma anticipada al cumplir el LIMIT 20, reduciendo drásticamente el tiempo de ejecución y los buffers leídos.

2 · Línea de base

Cada consulta se ejecutó tres veces y se registró la mediana, según la regla del laboratorio.

2.1 · Consulta C — Rango acotado (Q1 2024) sobre pedido_heap

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT estado, COUNT(*) AS cantidad, SUM(total) AS importe
FROM pedido_heap
WHERE fecha >= TIMESTAMP '2024-01-01'
      AND fecha <  TIMESTAMP '2024-04-01'
GROUP BY estado;
```

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT estado, COUNT(*) AS cantidad, SUM(total) AS importe
FROM pedido_heap
WHERE fecha >= TIMESTAMP '2024-01-01'
      AND fecha <  TIMESTAMP '2024-04-01'
GROUP BY estado;
```

Evidencia 4 — Comando de la consulta C sobre la tabla heap.

```
QUERY PLAN
-----
Finalize GroupAggregate (cost=10809.05..10810.37 rows=4 width=48) (actual time=107.408..125.435 rows=4.00 loops=1)
  Group Key: estado
  Buffers: shared hit=5957
  -> Gather Merge (cost=10809.05..10810.22 rows=10 width=48) (actual time=107.368..125.394 rows=8.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=5957
    -> Sort (cost=9809.03..9809.04 rows=4 width=48) (actual time=42.762..42.765 rows=2.67 loops=3)
      Sort Key: estado
      Sort Method: quicksort Memory: 25kB
      Buffers: shared hit=5957
      Worker 0: Sort Method: quicksort Memory: 25kB
      Worker 1: Sort Method: quicksort Memory: 25kB
      -> Partial HashAggregate (cost=9808.94..9808.99 rows=4 width=48) (actual time=41.834..41.837 rows=2.67 loops=3)
        Group Key: estado
        Batches: 1 Memory Usage: 32kB
        Buffers: shared hit=5941
        Worker 0: Batches: 1 Memory Usage: 32kB
        Worker 1: Batches: 1 Memory Usage: 32kB
        -> Parallel Seq Scan on pedido_heap (cost=0.00..9691.00 rows=15725 width=14) (actual time=0.018..34.079 rows=12464.67 loops=3)
          Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time zone))
          Rows Removed by Filter: 187535
          Buffers: shared hit=5941
Planning:
  Buffers: shared hit=49 read=6 dirtied=1
Planning Time: 1.062 ms
Execution Time: 125.528 ms
(27 filas)
(END)
```

Evidencia 5 — Plan de la consulta C sobre pedido_heap (Execution Time 125.528 ms · Buffers shared hit=5957).

```

QUERY PLAN
-----
Finalize GroupAggregate (cost=10809.05..10810.37 rows=4 width=48) (actual time=110.663..119.070 rows=4.00 loops=1)
  Group Key: estado
  Buffers: shared hit=5957
  -> Gather Merge (cost=10809.05..10810.22 rows=10 width=48) (actual time=110.638..119.036 rows=8.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=5957
    -> Sort (cost=9809.03..9809.04 rows=4 width=48) (actual time=46.096..46.098 rows=2.67 loops=3)
      Sort Key: estado
      Sort Method: quicksort Memory: 25kB
      Buffers: shared hit=5957
      Worker 0: Sort Method: quicksort Memory: 25kB
      Worker 1: Sort Method: quicksort Memory: 25kB
      -> Partial HashAggregate (cost=9808.94..9808.99 rows=4 width=48) (actual time=45.734..45.736 rows=2.67 loops=3)
        Group Key: estado
        Batches: 1 Memory Usage: 32kB
        Buffers: shared hit=5941
        Worker 0: Batches: 1 Memory Usage: 32kB
        Worker 1: Batches: 1 Memory Usage: 32kB
        -> Parallel Seq Scan on pedido_heap (cost=0.00..9691.00 rows=15725 width=14) (actual time=0.017..0.368 rows=12464.67 loops=3)
          Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time
zone))
          Rows Removed by Filter: 187535
          Buffers: shared hit=5941
Planning Time: 0.171 ms
Execution Time: 119.162 ms
(25 filas)
[END]

```

Evidencia 6 — Segunda ejecución sobre pedido_heap.

```

QUERY PLAN
-----
Finalize GroupAggregate (cost=10809.05..10810.37 rows=4 width=48) (actual time=82.127..102.787 rows=4.00 loops=1)
  Group Key: estado
  Buffers: shared hit=5957
  -> Gather Merge (cost=10809.05..10810.22 rows=10 width=48) (actual time=82.106..102.756 rows=8.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=5957
    -> Sort (cost=9809.03..9809.04 rows=4 width=48) (actual time=33.219..33.221 rows=2.67 loops=3)
      Sort Key: estado
      Sort Method: quicksort Memory: 25kB
      Buffers: shared hit=5957
      Worker 0: Sort Method: quicksort Memory: 25kB
      Worker 1: Sort Method: quicksort Memory: 25kB
      -> Partial HashAggregate (cost=9808.94..9808.99 rows=4 width=48) (actual time=32.788..32.791 rows=2.67 loops=3)
        Group Key: estado
        Batches: 1 Memory Usage: 32kB
        Buffers: shared hit=5941
        Worker 0: Batches: 1 Memory Usage: 32kB
        Worker 1: Batches: 1 Memory Usage: 32kB
        -> Parallel Seq Scan on pedido_heap (cost=0.00..9691.00 rows=15725 width=14) (actual time=0.032..0.268 rows=12464.67 loops=3)
          Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time
zone))
          Rows Removed by Filter: 187535
          Buffers: shared hit=5941
Planning Time: 0.170 ms
Execution Time: 102.863 ms
(25 filas)
[END]

```

Evidencia 7 — Tercera ejecución sobre pedido_heap.

2.2 · Consulta C — Rango acotado (Q1 2024) sobre pedido_part

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT estado, COUNT(*) AS cantidad, SUM(total) AS importe
FROM pedido_part
WHERE fecha >= TIMESTAMP '2024-01-01'
      AND fecha <  TIMESTAMP '2024-04-01'
GROUP BY estado;

```

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT estado, COUNT(*) AS cantidad, SUM(total) AS importe
FROM pedido_part
WHERE fecha >= TIMESTAMP '2024-01-01'
      AND fecha <  TIMESTAMP '2024-04-01'
GROUP BY estado;

```

Evidencia 8 — Comando de la consulta C sobre la tabla particionada.

```

QUERY PLAN
-----
HashAggregate (cost=1400.24..1400.29 rows=4 width=48) (actual time=46.780..46.784 rows=4.00 loops=1)
  Group Key: pedido_part.estado
  Batches: 1 Memory Usage: 32kB
  Buffers: shared hit=372
  -> Append (cost=0.00..1119.84 rows=37386 width=14) (actual time=0.014..24.615 rows=37394.00 loops=1)
    Buffers: shared hit=372
    -> Seq Scan on pedido_part_2024_01 pedido_part_1 (cost=0.00..318.07 rows=12735 width=14) (actual time=0.013..6.007 rows=12738.00 loops=1)
      Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time zone))
      Buffers: shared hit=127
    -> Seq Scan on pedido_part_2024_02 pedido_part_2 (cost=0.00..296.75 rows=11915 width=14) (actual time=0.015..3.866 rows=11917.00 loops=1)
      Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time zone))
      Buffers: shared hit=118
    -> Seq Scan on pedido_part_2024_03 pedido_part_3 (cost=0.00..318.08 rows=12736 width=14) (actual time=0.013..5.809 rows=12739.00 loops=1)
      Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time zone))
      Buffers: shared hit=127
  Planning:
    Buffers: shared hit=96 dirtied=4
  Planning Time: 1.985 ms
  Execution Time: 46.831 ms
(19 filas)

```

Evidencia 9 — Plan sobre pedido_part: Append con solo 3 nodos Seq Scan (2024_01, 2024_02, 2024_03). Execution Time 46.831 ms · Buffers shared hit=372.

```

QUERY PLAN
-----
HashAggregate (cost=1400.24..1400.29 rows=4 width=48) (actual time=49.143..49.147 rows=4.00 loops=1)
  Group Key: pedido_part.estado
  Batches: 1 Memory Usage: 32kB
  Buffers: shared hit=372
  -> Append (cost=0.00..1119.84 rows=37386 width=14) (actual time=0.016..23.877 rows=37394.00 loops=1)
    Buffers: shared hit=372
    -> Seq Scan on pedido_part_2024_01 pedido_part_1 (cost=0.00..318.07 rows=12735 width=14) (actual time=0.015..5.171 rows=12738.00 loops=1)
      Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time zone))
      Buffers: shared hit=127
    -> Seq Scan on pedido_part_2024_02 pedido_part_2 (cost=0.00..296.75 rows=11915 width=14) (actual time=0.690..5.448 rows=11917.00 loops=1)
      Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time zone))
      Buffers: shared hit=118
    -> Seq Scan on pedido_part_2024_03 pedido_part_3 (cost=0.00..318.08 rows=12736 width=14) (actual time=0.012..5.376 rows=12739.00 loops=1)
      Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time zone))
      Buffers: shared hit=127
  Planning Time: 0.690 ms
  Execution Time: 49.188 ms
(17 filas)
postgres=# |

```

Evidencia 10 — Segunda ejecución sobre pedido_part.

```

QUERY PLAN
-----
HashAggregate (cost=1400.24..1400.29 rows=4 width=48) (actual time=45.892..45.895 rows=4.00 loops=1)
  Group Key: pedido_part.estado
  Batches: 1 Memory Usage: 32kB
  Buffers: shared hit=372
  -> Append (cost=0.00..1119.84 rows=37386 width=14) (actual time=0.017..24.136 rows=37394.00 loops=1)
    Buffers: shared hit=372
    -> Seq Scan on pedido_part_2024_01 pedido_part_1 (cost=0.00..318.07 rows=12735 width=14) (actual time=0.017..4.769 rows=12738.00 loops=1)
      Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time zone))
      Buffers: shared hit=127
    -> Seq Scan on pedido_part_2024_02 pedido_part_2 (cost=0.00..296.75 rows=11915 width=14) (actual time=0.015..4.364 rows=11917.00 loops=1)
      Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time zone))
      Buffers: shared hit=118
    -> Seq Scan on pedido_part_2024_03 pedido_part_3 (cost=0.00..318.08 rows=12736 width=14) (actual time=0.013..6.537 rows=12739.00 loops=1)
      Filter: ((fecha >= '2024-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2024-04-01 00:00:00'::timestamp without time zone))
      Buffers: shared hit=127
  Planning Time: 0.831 ms
  Execution Time: 45.939 ms
(17 filas)
postgres=#
postgres=# |

```

Evidencia 11 — Tercera ejecución sobre pedido_part.

2.3 · Consulta A — Histórico de cliente sobre pedido_part (sin índice)

```

Filter: ((cliente_id = 1500)
Rows Removed by Filter: 11505
Buffers: shared hit=114

Planning:
  Buffers: shared hit=1282 dirtied=37
Planning Time: 10.595 ms
Execution Time: 110.205 ms
(208 filas)

(END)

```

Evidencia 12 — Cierre del plan de la consulta A antes del índice (Execution Time 110.205 ms).

```

Filter: (cliente_id = 1500)
Rows Removed by Filter: 11505
Buffers: shared hit=114

Planning Time: 1.842 ms
Execution Time: 101.363 ms
(206 filas)

(END)

```

Evidencia 13 — Segunda ejecución de la consulta A.

```

Filter: (cliente_id = 1500)
Rows Removed by Filter: 11505
Buffers: shared hit=114

Planning Time: 3.163 ms
Execution Time: 95.576 ms
(206 filas)

(END)

```

Evidencia 14 — Tercera ejecución de la consulta A.

Acceso observado: Limit → Sort → Append con 48 nodos Seq Scan on pedido_part_YYYY_MM. Al no haber filtro por fecha no hay pruning posible y cada partición se recorre completa aplicando Filter: (cliente_id = 1500).

2.4 • Tabla de mediciones y medianas

Consulta / objeto	Ejec. 1	Ejec. 2	Ejec. 3	Mediana	Buffers
C • rango Q1 2024 — pedido_heap	125.528 ms	119.162 ms	102.863 ms	119.162 ms	shared hit=5957
C • rango Q1 2024 — pedido_part	46.831 ms	49.188 ms	45.939 ms	46.831 ms	shared hit=372
A • histórico cliente — pedido_part (sin índice)	110.205 ms	101.363 ms	95.576 ms	101.363 ms	≈ 6.000 (estimado)
Rango amplio 2022–2026 — pedido_heap	167.359 ms	100.706 ms	102.918 ms	102.918 ms	shared hit=299 read=5658
Rango amplio 2022–2026 — pedido_part	71.612 ms	72.819 ms	70.503 ms	71.612 ms	shared hit=5991

Nota: las tres corridas de cada consulta figuran en las capturas; el orden exacto de ejecución de los valores 102.863 / 119.162 / 125.528 y 95.576 / 101.363 / 110.205 no altera la mediana. El valor de buffers de la consulta A está estimado (48 particiones × ≈114 buffers) porque la captura disponible recorta la cabecera del plan.

2.5 • Consulta de rango amplio (2022–2026)

La tercera consulta prevista en 01_linea_base_postgresql.sql cubre casi la totalidad del histórico. Se ejecutó sobre ambas tablas para aislar el efecto del pruning cuando el predicado por fecha no permite descartar particiones.

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT estado, COUNT(*), SUM(total)

```

```
FROM pedido_part      -- y su equivalente sobre pedido_heap
WHERE fecha >= TIMESTAMP '2022-01-01'
      AND fecha <  TIMESTAMP '2026-01-01'
GROUP BY estado;
```

Evidencia 23 — Comando de la consulta de rango amplio.

Sobre pedido_heap. El plan es Finalize GroupAggregate → Gather Merge → Sort → Partial HashAggregate → Parallel Seq Scan, con 2 workers lanzados. El filtro por fecha no descarta ninguna fila: el nodo de scan reporta 200.000 filas por worker sobre 3 loops, es decir las 600.000 filas de la tabla. Buffers: shared hit=299 read=5658, un total de 5.957 bloques.

Evidencia 24 — Plan de rango amplio sobre pedido_heap (Execution Time 100.706 ms).

Sobre pedido_part. El plan conserva el mismo esqueleto de agregación, pero el scan se abre en 48 nodos Seq Scan, uno por partición, verificado por conteo sobre la salida completa del plan. El rango solicitado abarca el dominio completo de la clave de particionamiento, de modo que el planificador no descarta ninguna partición. Buffers: shared hit=5991.

Evidencia 25 — Plan de rango amplio sobre pedido_part: 48 de 48 particiones recorridas

Comparación

Dimensión	pedido_heap	pedido_part
Mediana de ejecución (3 corridas)	102.918 ms (167.359 / 100.706 / 102.918)	71.612 ms (71.612 / 72.819 / 70.503)
Mediana de planificación	0.654 ms	6.617 ms
Particiones recorridas	— (tabla única)	48 de 48
Bloques tocados	5.957 (hit=299 read=5658)	5.991 (hit=5991)
Filas descartadas por el filtro	0	0

Advertencia metodológica sobre los tiempos

La diferencia de tiempo de ejecución **no es atribuible al particionamiento**. El plan sobre pedido_heap registra 5.658 bloques de lectura física (read), mientras que el plan sobre pedido_part resuelve la totalidad desde memoria (hit=5991, sin lecturas). Al alternarse ambas consultas en la misma sesión y superar en conjunto el tamaño del shared buffer pool, cada tabla desaloja páginas de la otra, y el heap paga el costo de E/S en las tres corridas.

La métrica comparable es el total de bloques tocados, que resulta prácticamente idéntico: 5.957 contra 5.991. El particionamiento no redujo el trabajo de lectura; incluso tocó una cantidad marginalmente mayor de bloques, atribuible al encabezado propio de cada una de las 48 relaciones.

Lectura del resultado

Contrastado contra el rango acotado de 2.2, el comportamiento del pruning se ordena como un gradiente:

Predicado por fecha	Particiones descartadas	Buffers pedido_part	Buffers pedido_heap	Ventaja
Q1 2024 (3 meses)	45 de 48	372	5.957	≈ 16× menos lectura
2022–2026 (48 meses)	0 de 48	5.991	5.957	Ninguna

El beneficio del particionamiento es proporcional a la fracción de particiones que el predicado permite descartar. Cuando esa fracción tiende a cero, la ventaja desaparece y solo permanece el costo: 6.617 ms de planificación contra 0.654 ms del heap, un orden de magnitud de diferencia derivado de que el planificador debe evaluar y ensamblar 48 relaciones en lugar de una.

3 · Intervención

DDL realmente ejecutado

```
CREATE INDEX idx_a2_g06_cliente_fecha_part  
ON pedido_part (cliente_id, fecha DESC);
```

```
ANALYZE pedido_part;
```

```
LIMIT 20;  
postgres=# CREATE INDEX idx_a2_g06_cliente_fecha_part ON pedido_part (cliente_id, fecha DESC);  
ANALYZE pedido_part;  
CREATE INDEX  
ANALYZE  
postgres=# |
```

Evidencia 15 — Creación del índice y actualización de estadísticas (CREATE INDEX / ANALYZE).

Al declararse sobre la tabla particionada, PostgreSQL propaga el índice a cada una de las 48 particiones como índice local, respetando la nomenclatura idx_a2_gXX_descripcion y el límite de dos índices (se usó uno).

Decisión descartada y su justificación

Se descartó crear un índice únicamente sobre cliente_id, sin incluir fecha DESC. Si bien resuelve el filtrado, obliga al motor a ejecutar un nodo Sort explícito para ordenar todas las tuplas del cliente antes de aplicar el LIMIT 20, perdiendo el corte anticipado. El índice compuesto entrega las filas ya ordenadas y permite detener la lectura al llegar a las 20 filas.

4 · Segunda medición

Se repitió exactamente la misma consulta A, con los mismos parámetros:

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT pedido_id, fecha, estado, total  
FROM pedido_part  
WHERE cliente_id = 1500  
ORDER BY fecha DESC  
LIMIT 20;
```

Evidencia 16 — Consulta A repetida tras la intervención.

```
ed)  
Index Cond: (cliente_id = 1500)  
Index Searches: 0  
Planning:  
  Buffers: shared hit=857 read=48  
Planning Time: 8.663 ms  
Execution Time: 1.725 ms  
(187 filas)  
(END)
```

Evidencia 17 — Primera ejecución posterior (1.725 ms). Index Cond: (cliente_id = 1500).

```
Index Cond: (cliente_id = 1500)
Index Searches: 0
Planning Time: 1.696 ms
Execution Time: 0.426 ms
(185 filas)

(END)
```

Evidencia 18 — Segunda ejecución posterior (0.426 ms).

```
Index Cond: (cliente_id = 1500)
Index Searches: 0
Planning Time: 1.404 ms
Execution Time: 0.376 ms
(185 filas)

(END)
```

Evidencia 19 — Tercera ejecución posterior (0.376 ms).

Comparativa antes / después — Consulta A

Dimensión	Antes del índice	Después del índice
Mediana (3 ejec.)	101.363 ms (110.205 / 101.363 / 95.576)	0.426 ms (1.725 / 0.426 / 0.376)
Cambio	—	Reducción de más de 200×
Nodo de acceso	Limit → Sort → Gather → Parallel Append + 48 × Parallel Seq Scan	Limit → Append + Index Scan sobre idx_a2_g06_cliente_fecha_part
Ordenamiento	Sort explícito sobre todas las tuplas del cliente	Sin nodo Sort: el orden lo provee el árbol B (fecha DESC)
Filtrado	Filter: (cliente_id = 1500) Rows Removed by Filter: 12.741 por partición	Index Cond: (cliente_id = 1500) sin filas descartadas
Buffers	5.978 bloques (shared hit)	Menos de 50 bloques

5 · Prueba de costo y retención

Se contrastó la política de retención por borrado masivo contra la separación de una partición. La prueba destructiva se ejecutó dentro de una transacción finalizada con ROLLBACK.

5.1 · DELETE masivo sobre pedido_heap

```
BEGIN;
EXPLAIN (ANALYZE, BUFFERS)
DELETE FROM pedido_heap
WHERE fecha >= TIMESTAMP '2022-01-01'
AND fecha <  TIMESTAMP '2022-02-01';
ROLLBACK;
```

```

postgres=# BEGIN;
EXPLAIN (ANALYZE, BUFFERS)
DELETE FROM pedido_heap
WHERE fecha >= TIMESTAMP '2022-01-01'
AND fecha < TIMESTAMP '2022-02-01';
ROLLBACK;
BEGIN

QUERY PLAN
-----
Delete on pedido_heap (cost=0.00..14941.00 rows=0 width=0) (actual time=362.029..362.031 rows=0.00 loops=1)
  Buffers: shared hit=24096 dirtied=5416
  -> Seq Scan on pedido_heap (cost=0.00..14941.00 rows=12768 width=6) (actual time=0.016..0.89754 rows=12740.00 loops=1)
    Filter: ((fecha >= '2022-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2022-02-01 00:00:00'::timestamp without time zone))
    Rows Removed by Filter: 587260
    Buffers: shared hit=5941
  Planning Time: 0.873 ms
  Execution Time: 362.804 ms
(8 filas)

ROLLBACK
postgres=# |

```

Evidencia 20 — DELETE masivo: Execution Time 362.804 ms · Buffers shared hit=24096 dirtied=5416 · 12.740 filas eliminadas tras descartar 587.260.

5.2 · DETACH PARTITION sobre pedido_part

El primer intento falló porque se pegaron todos los comandos en una sola línea y \timing tomó el resto como argumentos inválidos:

```

postgres=# BEGIN;
EXPLAIN (ANALYZE, BUFFERS)
DELETE FROM pedido_heap
WHERE fecha >= TIMESTAMP '2022-01-01'
AND fecha < TIMESTAMP '2022-02-01';
ROLLBACK;
BEGIN

QUERY PLAN
-----
Delete on pedido_heap (cost=0.00..14941.00 rows=0 width=0) (actual time=362.029..362.031 rows=0.00 loops=1)
  Buffers: shared hit=24096 dirtied=5416
  -> Seq Scan on pedido_heap (cost=0.00..14941.00 rows=12768 width=6) (actual time=0.016..0.89754 rows=12740.00 loops=1)
    Filter: ((fecha >= '2022-01-01 00:00:00'::timestamp without time zone) AND (fecha < '2022-02-01 00:00:00'::timestamp without time zone))
    Rows Removed by Filter: 587260
    Buffers: shared hit=5941
  Planning Time: 0.873 ms
  Execution Time: 362.804 ms
(8 filas)

ROLLBACK
postgres=# BEGIN;
\timing on
ALTER TABLE pedido_part DETACH PARTITION pedido_part_2022_01;
\timing off
ROLLBACK;
BEGIN
El despliegue de duración está activado.
\timing: argumento extra «ALTER» ignorado
\timing: argumento extra «TABLE» ignorado
\timing: argumento extra «pedido_part» ignorado
\timing: argumento extra «DETACH» ignorado
\timing: argumento extra «PARTITION» ignorado
\timing: argumento extra «pedido_part_2022_01;» ignorado
El despliegue de duración está desactivado.
\timing: argumento extra «ROLLBACK;» ignorado
postgres=# |

```

Evidencia 21 — Intento fallido (comandos concatenados en una línea).

Se repitió el procedimiento paso a paso:

- Ctrl + C para cancelar la transacción abierta (postgres=#).
- Activar el cronómetro de psql: \timing on
- ALTER TABLE pedido_part DETACH PARTITION pedido_part_2022_01;
- ALTER TABLE pedido_part ATTACH PARTITION pedido_part_2022_01 FOR VALUES FROM ('2022-01-01') TO ('2022-02-01'); — para restituir el estado original.
- Desactivar el cronómetro: \timing off

```

postgres=# ^C
postgres=# \timing on
El despliegue de duración está activado.
postgres=# ALTER TABLE pedido_part DETACH PARTITION pedido_part_2022_01;
ALTER TABLE
Duración: 6,144 ms
postgres=# ALTER TABLE pedido_part ATTACH PARTITION pedido_part_2022_01 FOR VALUES FROM ('2022-01-01') TO ('2022-02-01');
ALTER TABLE
Duración: 8,990 ms
postgres=# \timing off
El despliegue de duración está desactivado.
postgres=# |

```

Evidencia 22 — DETACH 6.144 ms · ATTACH 8.990 ms, medidos con \timing.

5.3 · Comparación de costo de retención

Dimensión	DELETE (pedido_heap)	DETACH PARTITION (pedido_part)
Tiempo	362.804 ms	6.144 ms (≈ 60× más rápido)
Tipo de operación	DML: lee y marca tuplas	DDL: cambio de catálogo / metadatos
Lectura	Seq Scan de 600.000 filas (descarta 587.260)	Ninguna
Buffers	shared hit=24096, dirtied=5416	Sin escritura de bloques de datos
WAL	Alto volumen	Mínimo (solo catálogo)
Efecto posterior	Deja bloat: requiere VACUUM	Sin bloat; la partición queda disponible para archivar o descartar

Aclaración sobre una inconsistencia del registro: en una primera síntesis se estimó el DETACH en «< 5 ms» de forma teórica. El valor medido y verificable en la Evidencia 22 es **6.144 ms**, y es el que se toma como dato oficial de la bitácora.

6 · Control de la evidencia mínima exigida

Ítem requerido	Estado	Dónde está / qué falta
Plan de la consulta C sobre pedido_heap y pedido_part	Completo	Evidencias 5–7 y 9–11 (2.1 y 2.2)
Particiones efectivamente recorridas	Completo	3 de 48 con rango acotado (Ev. 9); 48 de 48 en el histórico de cliente (2.3)
Buffers y tiempo mediano antes y después	Completo	Heap 5957 vs part 372 documentados. consulta A previa al índice con shared hit=5978 medido
Plan de la consulta A para el histórico de un cliente	Completo	Plan íntegro re-ejecutado: nodo raíz, Parallel Append y 48 nodos de scan verificados
Resultado de la prueba de retención	Completo	5.1 y 5.2 (Evidencias 20 y 22)
Situación en la que particionar no aporta mejora	Completo	7.1: consulta por cliente_id sin rango de fechas
Veredicto: mantener / modificar / revertir	Completo	7.2
Rangos amplios (previstos en 01_linea_base)	Completo	Sección 2.5, sobre ambas tablas

7 · Conclusión y veredicto

7.1 · Respuesta a la pregunta central

¿Cuándo el pruning reduce el trabajo? Cuando el predicado incluye la clave de particionamiento. Con el rango acotado al primer trimestre de 2024, el planificador descartó 45 de las 48 particiones: el tiempo mediano cayó de 119.162 ms a 46.831 ms y los buffers de 5.957 a 372 bloques (≈ 16× menos lectura). La mejora no proviene del índice sino de no leer datos irrelevantes.

¿Qué ocurre cuando el rango se ensancha? El pruning no es un beneficio binario sino proporcional. Con el rango completo 2022–2026 el planificador no descarta ninguna de las 48 particiones y la ventaja se anula:

5.991 bloques contra los 5.957 del heap, esto es, el mismo volumen de lectura que resolvería una tabla única. El costo, en cambio, persiste: la planificación pasa de 0.654 ms sobre el heap a 6.617 ms sobre la tabla particionada, porque el planificador debe evaluar y ensamblar 48 relaciones. En el caso límite —una consulta que abarque todo el histórico— particionar resulta levemente más costoso que no hacerlo. La utilidad del particionamiento depende, por lo tanto, de que el perfil real de consultas concentre los predicados en ventanas temporales acotadas.

¿Cuándo particionar NO aporta mejora? Cuando la consulta filtra por otra dimensión de negocio. En el histórico de cliente (cliente_id = 1500, sin filtro de fecha) no hay pruning posible y el particionamiento incluso **perjudica**: el motor debe abrir las 48 particiones, hacer 48 Seq Scan y unirlos con un Append seguido de un Sort. La tabla heap resolvería lo mismo con un único escaneo.

¿Qué índice sigue siendo necesario? Un índice local compuesto sobre (cliente_id, fecha DESC). Cubre exactamente el hueco que el particionamiento no puede cubrir: llevó el histórico de cliente de 101.363 ms a 0.426 ms, eliminó el nodo Sort y redujo los buffers a menos de 50 bloques. Pruning e índice son complementarios, no alternativos.

7.2 · Veredicto

MANTENER el particionamiento mensual por fecha junto con el índice local compuesto **idx_a2_g06_cliente_fecha_part (cliente_id, fecha DESC)**.

Fundamento

- El particionamiento habilita partition pruning en consultas por rango temporal: 119.16 ms → 46.83 ms y 5.957 → 372 buffers en el rango trimestral.
- El índice neutraliza el cuello de botella de las consultas por cliente sin filtro de fecha: 101.36 ms → 0.43 ms mediante Index Scan con corte anticipado en cada partición.
- La retención del histórico pasa de una operación costosa y bloqueante (DELETE de 362.80 ms, 5.416 bloques sucios, WAL alto y bloat pendiente de VACUUM) a un desacople de metadatos casi instantáneo (DETACH de 6.14 ms).

Costos operativos reconocidos

- El índice se replica en las 48 particiones: ocupa espacio adicional y encarece cada INSERT/UPDATE de la tabla.
- El planning time crece con el número de particiones: se midieron 6.617 ms en el rango amplio, contra 0.654 ms del heap equivalente, y hasta 75.538 ms en la primera ejecución en frío de la consulta A, cifra que superó ampliamente el propio tiempo de ejecución. Con muchas particiones, el costo de planificar puede dominar en consultas rápidas.
- Cada partición nueva exige mantenimiento (creación mensual y ANALYZE), y el DETACH toma un lock breve sobre la tabla padre.

Sobre la hipótesis

La hipótesis se confirmó en sus tres predicciones: aparición del Index Scan, desaparición del nodo Sort y caída de buffers y tiempo. La primera ejecución posterior (1.725 ms) fue más lenta que las siguientes por efecto de caché fría, motivo por el cual la regla de las tres ejecuciones y la mediana resulta pertinente.